

The Recursive File Explorer

The Goal: Build a program that mimics a computer's file system. You must navigate through nested folders (dictionaries) to calculate the total storage size of files, even if those files are buried 10 levels deep.

1. Scenario for Students

You are a Data Engineer. Your company has a massive messy hard drive represented by a Python Dictionary. Some folders contain files (numbers representing size), and some folders contain *other* folders. Your boss needs a tool that calculates the total size of any folder the user selects, no matter how deep the subfolders go.

2. The Data Structure (Setup)

This represents the "Hard Drive."

Python

```
None
file_system = {
    "root": {
        "documents": {
            "resume.txt": 15,
            "budget.xls": 30,
            "old_stuff": {
                "letters": {
                    "love_letter.txt": 2,
                    "breakup_letter.txt": 2
                },
                "school": {
                    "homework.docx": 10
                }
            },
        },
        "photos": {
            "cat.jpg": 100,
            "dog.png": 105,
            "vacation": {
```

```
        "beach.png": 200,  
        "mountain.jpg": 200  
    },  
    },  
    "system_log.txt": 50  
}  
}
```

3. Technical Requirements

Part A: The Recursive Function (The Logic)

Create a function called `calculate_size(current_item)`.

- **Base Case (The Stop Condition):** If `current_item` is an Integer (a file), return the number.
- **Recursive Step:** If `current_item` is a Dictionary (a folder), use a `for` loop to iterate through its contents. Inside that loop, call `calculate_size()` on the sub-items and add the returned values to a total.

Part B: The Iterative Function (The Menu)

Create a function called `main_menu()`.

- Use a `while` loop to keep the program running.
- Ask the user to type the name of a folder they want to analyze (e.g., "photos" or "old_stuff").
- Since the user might not know the full path, you need a helper search function (iterative or recursive) to find *where* that folder exists in the big dictionary, or simply let them search the top-level keys for simplicity (or make the search part of the challenge).

Part C: Exception Handling

- If the user types a folder name that doesn't exist, handle the `KeyError` or return `None` and print "Folder not found."
- Ensure the program doesn't crash if the dictionary structure is unexpected.

4. Sample Expected Output

None

--- FILE SYSTEM SCANNER ---

Current Root Folders: ['documents', 'photos', 'system_log.txt']

Enter a folder name to scan (or 'exit'): photos

> Scanning 'photos'...

> Found file: cat.jpg (100MB)

> Found file: dog.png (105MB)

> Entering subfolder 'vacation'...

> Found file: beach.png (200MB)

> Found file: mountain.jpg (200MB)

TOTAL SIZE: 605 MB

Enter a folder name to scan: old_stuff

> Scanning 'old_stuff'...

> Entering subfolder 'letters'...

...

TOTAL SIZE: 14 MB

Enter a folder name to scan: alien_plans

Error: Folder 'alien_plans' does not exist in the current directory!